

Тема урока: декораторы @classmethod и @staticmethod

1. Методы экземпляра
2. Методы класса, декоратор @classmethod
3. Статические методы, декоратор @staticmethod

Аннотация. Урок посвящен декораторам @classmethod и @staticmethod .

Методы в Python

В Python есть несколько типов методов:

1. методы экземпляра
2. методы класса
3. статические методы

Методы экземпляра

Методы, с которыми мы познакомились в прошлых уроках, являлись методами экземпляра. Именно их мы используем большую часть времени. Как уже известно, методы экземпляра имеют обязательный параметр `self` , который указывает на конкретный экземпляр класса при вызове метода. Через переменную `self` методы экземпляра могут свободно обращаться к атрибутам объектов и полноценно с ними работать, что дает им возможность управлять их состоянием.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name, age):
        self.name = name           # имя кошки
        self.age = age             # возраст кошки
        self.favorite_things = []  # список любимых вещей
кошки

    def about(self):
        return f'Имя: {self.name}, возраст: {self.age}'
атрибутам объекта

    def loves(self, thing):
        self.favorite_things.append(thing)
атрибута объекта

cat = Cat('Кемаль', 1)

cat.loves('Тимур')
cat.loves('Дом')

print(cat.about())
print(cat.favorite_things)
```

ВЫВОДИТ:

```
Имя: Кемаль, возраст: 1
['Тимур', 'Дом']
```

Метод экземпляра также можно вызвать через класс, передав в качестве первого аргумента сам экземпляр класса.

Приведенный ниже код:

```
cat = Cat('Кемаль', 1)

cat.loves('Тимур')           # вызов метода через
экземпляр
Cat.loves(cat, 'Дом')        # вызов метода через
класс

print(Cat.about(cat))        # вызов метода через
класс
print(cat.favorite_things)
```

ВЫВОДИТ:

```
Имя: Кемаль, возраст: 1
['Тимур', 'Дом']
```

Методы класса

Методы класса, как следует из названия, тесно связаны с классом, в котором они определены. В отличие от методов экземпляра, имеющих обязательный параметр `self`, указывающий на экземпляр класса, методы класса имеют обязательный параметр `cls`, который указывает на сам класс. Такие методы имеют возможность менять состояние класса, что в свою очередь отражается на всех экземплярах данного класса. Однако менять состояние конкретного объекта как это делают методы экземпляра, они не могут, так как имеют доступ лишь к классу.



Присутствие параметра `cls` в методе класса обязательное. Мы не обязаны называть его `cls`, однако строго рекомендуется действовать согласно общепринятой практике.

Для создания методов класса используется декоратор `@classmethod`.

Приведенный ниже код:

```
class MyClass:
    @classmethod
    def my_classmethod(cls):
        print('Это метод класса')
        print(cls)

MyClass.my_classmethod()
```

создает в классе `MyClass` метод класса `my_classmethod()` и выводит:

```
Это метод класса
<class '__main__.MyClass'>
```

Как мы видим, метод класса `my_classmethod()` действительно имеет доступ лишь к классу `MyClass`.

Зачастую методы класса используются для добавления альтернативного способа создания экземпляров класса. Рассмотрим следующее определение класса `Cat`:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed                # порода кошки
        self.name = name                  # имя кошки
```

Экземпляры данного класса имеют два атрибута — породу и имя. Скажем, мы заметили, что очень часто имеем дело с кошками британской породы и решили, что было бы удобно создавать объекты класса `Cat`, указывая лишь имя, а в качестве породы по умолчанию устанавливая `Британский`. Для этого мы можем реализовать метод класса, который принимает лишь имя кошки и возвращает объект класса `Cat` британской породы с указанным именем.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

    @classmethod
    def british(cls, name):
        return cls('Британский', name) # равнозначно
Cat('Британский', name)

cat = Cat.british('Кемаль')

print(cat.breed, cat.name)
```

ВЫВОДИТ:

Британский Кемаль

На самом деле мы уже не раз сталкивались с методами класса, например, когда изучали такие типы данных как `dict` и `datetime`.

Приведенный ниже код:

```
from datetime import datetime

cats = dict.fromkeys(['Кемаль', 'Роджер']) # словарь со
значениями по умолчанию
dt = datetime.strptime('12.10.2022', '%d.%m.%Y') # дата на основе
строки

print(cats)
print(dt)
```

ВЫВОДИТ:

```
{'Кемаль': None, 'Роджер': None}
2022-10-12 00:00:00
```

Статические методы

Статические методы, в отличие от методов экземпляра и методов класса, не имеют обязательного параметра `self` или `cls`. Поэтому статические методы не могут изменять ни состояние объекта, ни состояние класса. Статические методы можно считать обычными функциями, которые помещены в класс для удобства. Чаще всего это какой-то вспомогательный код, предназначенный для внутреннего использования.

Для создания статических методов используется декоратор `@staticmethod`.

Приведенный ниже код:

```
class MyClass:
    @staticmethod
    def my_staticmethod():
        print('Это статический метод')

MyClass.my_staticmethod()
```

создает в классе `MyClass` статический метод класса `my_staticmethod()` и выводит:

Это статический метод

Как мы видим, статический метод `my_staticmethod()` действительно не имеет доступа ни к классу, ни к объекту.

Статические методы могут вызываться внутри методов экземпляра или класса для вычисления каких-либо значений, которые напрямую не связаны с экземплярами класса или самим классом. Рассмотрим следующее определение класса `Cat` :

```
class Cat:
    def __init__(self, name, age):
        self.name = name                # имя кошки
        self.age = age                  # возраст кошки
```

Например, мы хотим определить метод `get_age()` , который мог бы возвращать возраст кошки как в кошачьих годах, так и в человеческих. Тогда мы можем определить статический метод `age_in_human_years()` , который переводит кошачьи года в человеческие, и использовать его внутри метода `get_age()` .

Приведенный ниже код:

```

class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_age(self, in_human_years=False):
        if in_human_years:
            return Cat.age_in_human_years(self.age) # переводим кошачьи
года в человеческие
        return self.age

    @staticmethod
    def age_in_human_years(age):
        return (age + (age < 5) - (age > 8)) * 7 # переводим кошачьи
года в человеческие

cat = Cat('Кемаль', 2)

print(cat.get_age()) # возраст в кошачьих
годах
print(cat.get_age(True)) # возраст в
человеческих годах

```

ВЫВОДИТ:

```

2
21

```

Примечания

Примечание 1. Помимо того, что методы экземпляра могут изменять состояние объекта, они так же могут получать доступ к самому классу через атрибут `__class__` экземпляра класса. Это означает, что методы экземпляра также могут изменять состояние класса.

Приведенный ниже код:

```

class MyClass:
    def method(self):
        print('Это метод экземпляра')
        print(self)
        print(self.__class__)

obj = MyClass()

obj.method()

```

ВЫВОДИТ (адрес может отличаться):

```

Это метод экземпляра
< main .MyClass object at 0x000002733A5A6BC0>

```



